

Two-Stage Time-Step Refinement for GRAPE

Abstract

I reduced the GRAPE time step by a factor of ten with a two-stage schedule. Stage 1 fits a multi-truncation pulse on the base grid $dt = 2$ ns. Stage 2 warm-starts from that pulse, upsamples by $s = 10$, and refines with `refine_pulse_dt_light` on a single truncation $n_c = 24$, with a halfway multi-truncation check. For U_{opt} and U_{enc} , plateau fidelities rise to ≈ 0.999 and ≈ 0.9997 at $dt = 0.2$ ns. This note documents the schedule, compares the two functions, and reports fidelity, photon trajectories, and cost.

1 Why shrink dt

The pulses in this project are piecewise constant: N steps of width dt . A smaller dt is a better control approximation—less zero-order-hold error, more temporal freedom for L-BFGS-B. The total duration $T = N dt$ stays fixed, so shrinking dt by an integer factor s means

$$N \leftarrow sN, \quad dt \leftarrow dt/s. \quad (1)$$

The catch is cost. Each fidelity and gradient evaluation diagonalizes a joint Hamiltonian at every time step, so

$$T_{\text{eval}} \propto N (n_t n_c)^3. \quad (2)$$

A factor-of-ten finer grid costs about ten times more per evaluation. Multi-truncation training multiplies that again. Doing both on every iteration is expensive.

The method I used is a two-stage split:

1. **Stage 1.** Fit on the coarse grid with multi-truncation training via `optimize_multi_state_pulse`.
2. **Stage 2.** Upsample and re-optimize on a single truncation via `refine_pulse_dt_light`, checking truncation only at halfway and at the end.

I show results for two pulses: $U_{\text{opt}} (|g, 0\rangle \rightarrow |g, 6\rangle)$ and U_{enc} (cat-code encode).

2 Two-stage schedule

2.1 Stage 1: multi-truncation fit

Stage 1 calls `optimize_multi_state_pulse` on the base grid $dt = 2$ ns. Every iteration averages fidelity over a truncation list (e.g. $\{22, 24, 26\}$), optionally with the discrepancy penalty so the three truncations stay in agreement. The point is a pulse whose $F(n_c)$ is already flat—so Stage 2 does not inherit wall exploitation.

Pulses used here:

- `u_opt_main.npy`: shape (250, 4), $0.5 \mu\text{s}$.
- `u_enc_main.npy`: shape (550, 4), $1.1 \mu\text{s}$.

2.2 Stage 2: light dt refinement

Stage 2 calls `refine_pulse_dt_light` with $s = 10$:

1. **Upsample** by zero-order hold: each row of u is repeated s times. Duration is unchanged; dt becomes 0.2 ns.
2. **Train** on a single truncation $n_c = 24$ (one Hamiltonian, no multi-trunc loop per iteration).
3. **Halfway check** at iteration $\approx \text{maxiter}/2$: sweep F over $n_c \in \{18, \dots, 30\}$ without restarting L-BFGS-B.
4. **Final check**: same sweep after optimization.

Outputs: `u_opt10x_light.npy` ($N = 2500$) and `u_enc10x_light.npy` ($N = 5500$). Logs: `light_refine_10x_run.log`, `light_refine_enc10x_run.log`.

There is also a heavier path, `refine_pulse_dt`, which upsamples the same way but then runs full multi-truncation on the fine grid. That is correct but much more expensive. Everything below uses the light path.

2.3 How the two functions differ

Table 1 is the short version. They are complementary: Stage 1 finds a truncation-safe pulse; Stage 2 only polishes dt . The light refiner assumes the warm start already has convergent $F(n_c)$. If Stage 1 failed that test, single-truncation fine training could reintroduce wall exploitation. The halfway sweep is a safety net, not a substitute for multi-truncation discovery.

Table 1: Stage 1 optimizer vs Stage 2 light refiner.

	<code>optimize_multi_state_pulse</code>	<code>refine_pulse_dt_light</code>
Role	Truncation-robust fit on (N, dt)	dt polish of a robust warm start
Time grid	Caller sets N, dt	Upsample by s : $N \rightarrow sN$, $dt \rightarrow dt/s$
Per-iteration trunc.	Full <code>trunc_list</code>	Single n_c
Parallelism	<code>joblib</code> over truncations	None
Discrepancy (Eq. 24)	Optional	Not used
Trunc. monitoring	Inside the objective	Mid-run + final checks only
Warm start	Optional	Required (ZOH-upsampled Stage 1)
Cost per eval	$\propto \text{trunc_list} N n^3$	$\propto s N n^3$ at one n_c

Both paths share the batched fidelity core, which amortizes the per-step eigendecomposition across state pairs and batches `eigh` in chunks of 256. That is what makes large- N (large- s) evaluations practical.

3 Fidelity vs cavity truncation

Figure 1 and Table 2 compare Stage 1 (`*_main.npy`, $dt = 2$ ns) with Stage 2 (`*10x_light.npy`, $dt = 0.2$ ns) over $n_c \in \{18, 20, \dots, 30\}$.

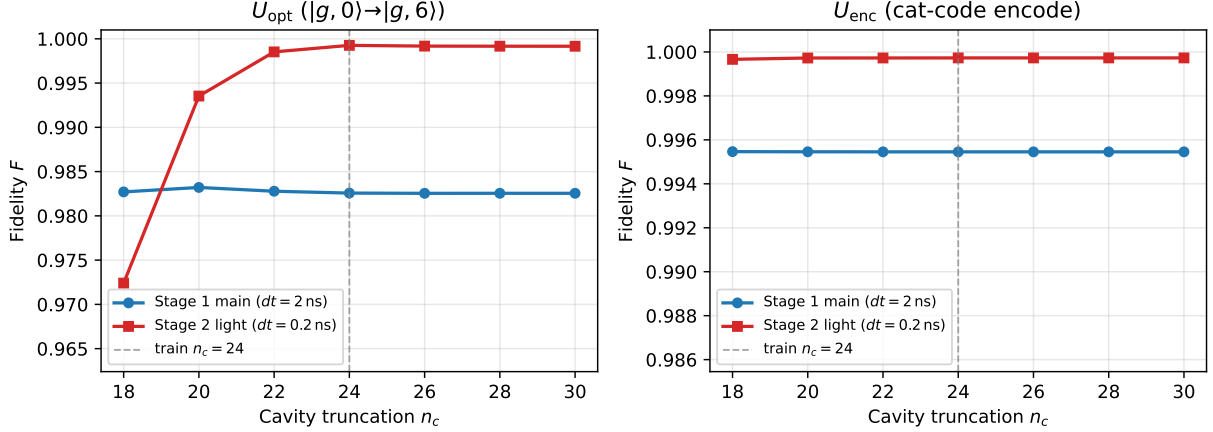


Figure 1: Fidelity vs cavity truncation. Dashed line: Stage 2 training truncation $n_c = 24$.

Table 2: Fidelity $F(n_c)$. $\Delta F = F_{\text{light}} - F_{\text{main}}$.

n_c	U_{opt}			U_{enc}	
	F_{main}	F_{light}	ΔF	F_{main}	F_{light}
18	0.9827	0.9724	-0.0103	0.9955	0.9997
20	0.9832	0.9935	+0.0103	0.9955	0.9997
22	0.9828	0.9985	+0.0157	0.9955	0.9997
24	0.9826	0.9993	+0.0167	0.9955	0.9997
26	0.9825	0.9992	+0.0166	0.9955	0.9997
28	0.9825	0.9992	+0.0166	0.9955	0.9997
30	0.9825	0.9992	+0.0166	0.9955	0.9997

U_{opt} . Stage 1 already plateaus at $F \approx 0.983$ —truncation-convergent, but limited by the coarse grid. Stage 2 lifts that to $F \gtrsim 0.999$ for $n_c \geq 22$ ($\Delta F \approx +0.017$). At $n_c = 18$ the fine pulse is slightly worse (0.972 vs 0.983); the halfway log check showed the same dip. That is low-truncation sensitivity, not wall exploitation (which would collapse just above the training truncation). At and above $n_c = 24$, F is flat and high.

U_{enc} . Both stages are flat across the full sweep. Stage 2 gains a nearly constant $\Delta F \approx +4.3 \times 10^{-3}$ (0.9955 $\rightarrow$ 0.9997). Halfway and final checks agree: $F \geq 0.9996$ at every n_c tested.

Both Stage 2 runs hit the 2000-iteration limit rather than a gradient tolerance, so these fidelities are lower bounds on a longer budget.

4 Photon trajectories

Figure 2 shows $\langle n \rangle(t)$ for both pulses at $n_c = 24$, Stage 1 vs Stage 2 side by side (initial state $|g, 0\rangle$; encode uses the ground branch). Table 3 has the scalar metrics. Fock heatmaps $P(n, t)$ are in Appendix A.

For U_{opt} , both stages finish at $\langle n \rangle \approx 6$. The light pulse uses a higher intermediate $\langle n \rangle$ (≈ 9.3 vs ≈ 7.9) and hits the amplitude threshold (40 rad/ μ s). Final transmon leakage drops from $\sim 5 \times 10^{-3}$ to $\sim 2 \times 10^{-4}$.

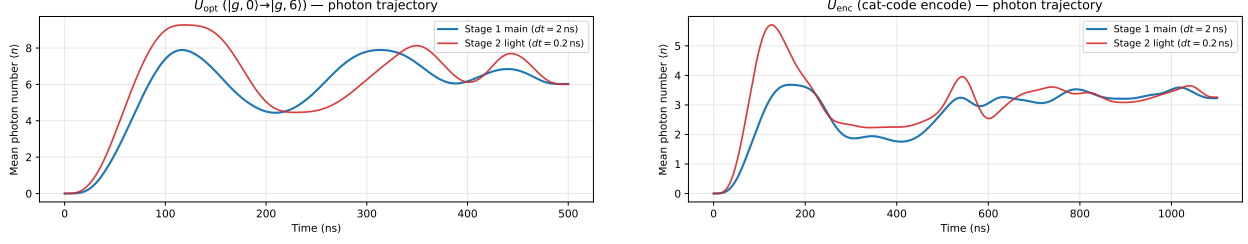


Figure 2: Mean photon number $\langle n \rangle(t)$ at $n_c = 24$. Left: U_{opt} (both stages end near $\langle n \rangle = 6$). Right: U_{enc} , $|g, 0\rangle$ branch.

Table 3: Trajectory and amplitude metrics at $n_c = 24$. Amplitudes in $\text{rad}/\mu\text{s}$.

Pulse	Stage	N	peak $ u $	max $\langle n \rangle$	final $\langle n \rangle$	max $P_{\text{tr. ex}}$
opt	main	250	36.69	7.89	6.024	0.807
opt	light	2500	40.00	9.27	6.004	0.723
enc	main	550	20.63	3.68	3.228	0.581
enc	light	5500	29.62	5.71	3.261	0.666

For U_{enc} , Stage 2 again takes a higher intermediate path and finishes near the cat-code mean $\sim |\alpha|^2 = 3$. Final transmon excitation is $\sim 6 \times 10^{-5}$. Photon support stays well below $n_c = 24$ —no pile-up at the Hilbert-space edge.

5 Cost as s scales

With duration fixed, $N(s) = sN_0$, so $T_{\text{eval}} \propto s$ at fixed Hilbert-space dimension. Full multi-truncation Stage 2 would multiply CPU work by roughly $\sum_k (n_{c,k}/n_c^*)^3$ again (about $3.2\times$ for $\{20, 24, 28\}$ relative to $n_c = 24$ alone). Light refinement drops that factor from the training loop and only pays multi-trunc cost at the two checkpoint sweeps.

From the logs ($s = 10$, $n_c = 24$, 2000 iterations):

- U_{opt} : $N' = 2500$, wall clock 4143 s (≈ 69 min), $F_{24} = 0.99926$.
- U_{enc} : $N' = 5500$, wall clock 10472 s (≈ 175 min), $F_{24} = 0.99973$.

The N ratio is 2.2; the wall-time ratio is ≈ 2.53 . The excess matches multi-state overhead ($M = 2$ encode pairs vs $M = 1$ Fock prep) plus line-search variability—not a breakdown of $T \propto N$. Under pure $T \propto s$ scaling, $s = 2, 5, 10$ would cost 1 : 2.5 : 5 relative to an $s = 1$ warm-start refine at the same n_c . Only $s = 10$ was run; the linear model is the design assumption.

So the schedule puts multi-truncation on the cheap coarse grid, and the fine grid on a single truncation.

6 Discussion

The split works as intended:

1. Stage 1 gives flat $F(n_c)$ on the coarse grid.
2. Stage 2 buys a clear fidelity gain at $dt/10$ without losing that flatness near and above $n_c = 24$.

3. Wall times at $s = 10$ stay in the one-to-three-hour range for 2000 iterations.

`refine_pulse_dt_light` is a refinement tool, not a replacement for multi-truncation training. The halfway check is the operational safeguard: stable early for U_{enc} ; mild $n_c = 18$ dip for U_{opt} , with no collapse above $n_c = 24$.

Caveats worth flagging: both Stage 2 solves hit the iteration limit; peak amplitude grows on the fine grid (opt reaches 40 rad/ μ s); only opt and enc were refined at $s = 10$, though the same schedule applies to the rest of the gate suite.

7 Conclusion

I reduced dt by a factor of ten with multi-truncation fitting on $dt = 2$ ns, then light single-truncation polish at $n_c = 24$, $s = 10$. Fidelity plateaus rise from ≈ 0.983 to ≈ 0.999 (U_{opt}) and from ≈ 0.9955 to ≈ 0.9997 (U_{enc}). Cost scales as expected with $N = sN_0$. Stage 1 buys truncation safety; Stage 2 spends it on a smaller dt .

References

- [1] R. W. Heeres, P. Reinhold, N. Ofek, *et al.*, Implementing a universal gate set on a logical qubit encoded in an oscillator, *Nat. Commun.* **8**, 94 (2017).
- [2] N. Khaneja, T. Reiss, C. Kehlet, T. Schulte-Herbrüggen, and S. J. Glaser, Optimal control of coupled spin dynamics: design of NMR pulse sequences by gradient ascent algorithms, *J. Magn. Reson.* **172**, 296–305 (2005).

A Fock population heatmaps

Full $P(n, t)$ for Stage 1 vs Stage 2 at $n_c = 24$. Red curve: $\langle n \rangle(t)$. These support the trajectory discussion in Sec. 4.

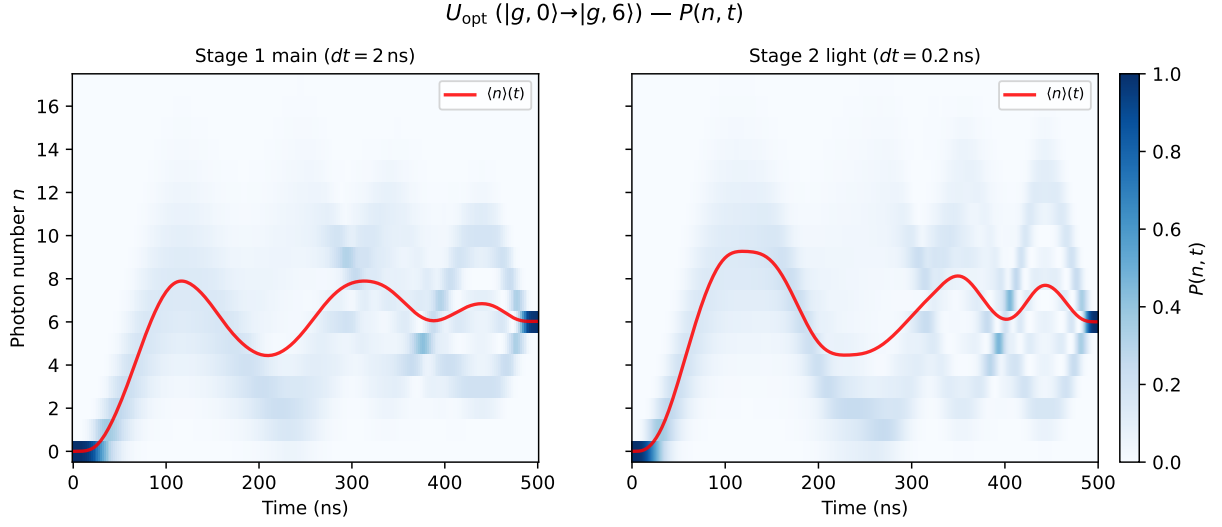


Figure 3: $P(n, t)$ for U_{opt} . Stage 2 uses a slightly more aggressive intermediate photon path but still ends on $|n = 6\rangle$.

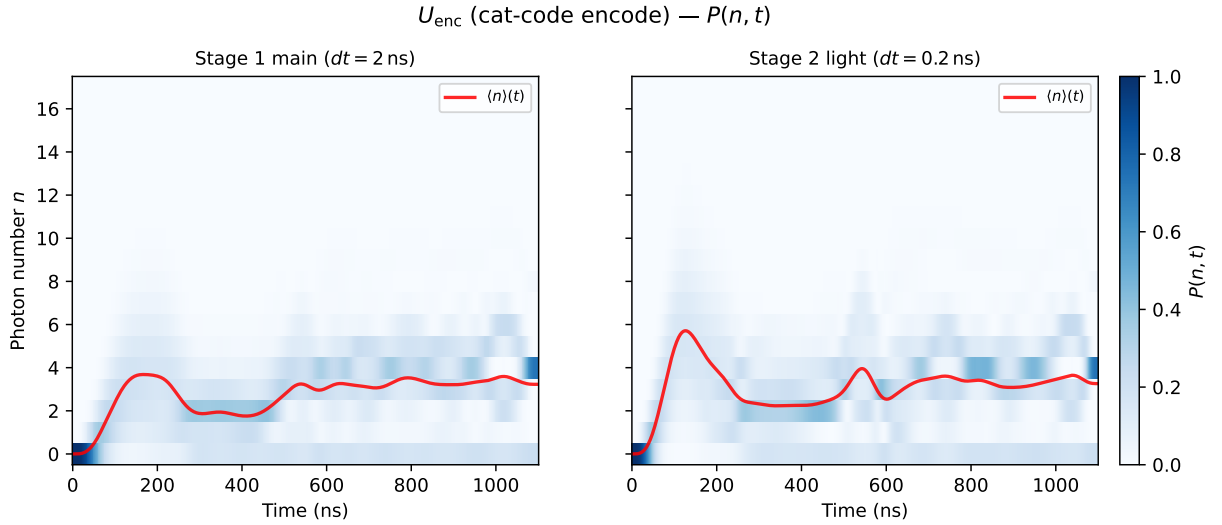


Figure 4: $P(n, t)$ for $U_{\text{enc}} (|g, 0\rangle \text{ branch})$. Population spreads over the even-parity support of the four-component cat ($\alpha = \sqrt{3}$).